Encapsulation

A lovely day at the bank



```
class BankAccount {

    int balance;

    BankAccount(int balance) {
        this.balance = balance;
    }

    boolean withdraw(int amount) {
        if (amount <= this.balance) {
            this.balance = this.balance - amount;
            return true;
        }
        return false;
    }

}
```

A lovely day at the bank

Meet Bob.

```
public class Main {

    public static void main(String[] args){
        BankAccount account = new BankAccount(100);
        account.withdraw(75);

        account.balance = 500_000;
    }

}
```



Bob works for North Korea.

What's the problem?

We want to ensure that our bank accounts satisfy certain **invariants**:

- E.g. an account cannot be overdrawn.
- E.g. a withdrawal is always deducted from the balance.
- E.g. a receipt is always printed when a withdrawal is made.
- …

We want:

To ensure principled access to the state of an object.

How can we do this?

Encapsulation

Key Idea:

- All state of an object should be private (i.e. inaccessible to the outside world.)
- Mutation should only be possible through methods which ensure integrity of the state.

(the state of an object = the value of its fields.)

Example

```
class BankAccount {

    private int balance;

    BankAccount(int balance) {
        this.balance = balance;
    }

    int withdraw(int amount) {
        if (amount <= this.balance) {
            this.balance = this.balance - amount;
            return true;
        }
        return false;
    }

}
```

Access Modifiers

Modifier	Class	Package	Subclass	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
`<<no modifier>>`	Y	Y	N	N
`private`	Y	N	N	N

Golden rule: *Everything should have the least visibility.*

Corollary: *Everything should be private by default.*

Getters and Setters

```
class Person {
    private String name;
    private int age;

    // constructor omitted.

    public String getName() {
        return this.name;
    }

    public int getAge() {
        return this.age;
    }

    public void setAge(int age) {
        if (age >= 0) {
            this.age = age;
        }
    }
}
```

Getters and Setters

```
class Temperature {
    private int celsius;

    // constructor omitted.

    public int getKelvin() {
        return /* ... */;
    }

    public int getCelsius() {
        return /* ... */;
    }

    public int getFahrenheit() {
        return /* ... */;
    }
}
```

What would happen if the program used the celsius field directly?